

ZT-XGuard: A Zero-Trust Framework for Continuous Trust Assessment and Secure Containment of Compromised O-RAN xApps

Janindu Dewmith*, Deshan Lokuge*, Kavinda Dilshan*, Thilohith Thiruchelvam*,
Pramitha Fernando†, Chatura Seneviratne* and An Braeken†

*Department of Electrical and Information Engineering, University of Ruhuna, Sri Lanka

†Vrije Universiteit Brussel, Brussels, Belgium

Abstract—The Open Radio Access Network (O-RAN) opens the mobile radio access network into interoperable, software-defined components from multiple vendors and its Near-Real-Time RAN Intelligent Controller (Near-RT RIC) makes the network programmable by hosting third-party applications called xApps. These xApps also widen the attack surface: once admitted, an xApp can be compromised and abuse compute, memory, network access or RIC control from inside the control loop. Admission-time checks and alert-only monitoring cannot tell whether a running xApp is still trustworthy, nor isolate it once it is compromised. We present ZT-XGuard, a zero-trust framework that continuously assesses each xApp’s trust and, on confirmed compromise, isolates it with enforcement that is verified rather than assumed. Detection combines two complementary channels targeting different threat classes: a kernel-level, extended Berkeley Packet Filter (eBPF) based behavioral channel that catches hard exploitation events and a statistical channel that flags abnormal resource use and confirms it before acting. Containment revokes the xApp’s cryptographic workload identity, cuts its network and service reachability through independent enforcement paths and then confirms the xApp is unreachable with an active probe before recording the incident as contained. On a Kubernetes-based Near-RT RIC testbed, ZT-XGuard contained all 400 behavioral attack trials with network cutoff in a median of 2.3 s (p95 4.2 s) and detected all evaluated resource-abuse scenarios with no false isolations across 4,397 benign samples.

Index Terms—O-RAN, Near-RT RIC, xApp, zero trust, anomaly detection, threat containment.

I. INTRODUCTION

O-RAN disaggregates the radio access network into multi-vendor components connected by open interfaces [1]. The Near-RT RIC hosts xApps that optimize radio resources, mobility, interference and quality of service over 10 ms–1 s control loops. This programmability creates a new threat surface: an xApp receives sensitive RIC telemetry, influences control decisions and runs as a cloud-native workload beside critical network functions. When a third-party xApp, or its image, runtime or granted permissions, is compromised, it becomes an insider on the RAN control plane.

The O-RAN Alliance documents security requirements for xApp access control, interface protection, image integrity and continuous monitoring [2], but a deployable framework must still answer three operational questions admission policy cannot: is a running xApp still trustworthy, what evidence declares

it compromised and once compromised, how does the platform verify that containment took effect rather than trusting a label or an alert? These matter because a malicious xApp rarely violates a static policy; it behaves normally, then drifts into a slow resource-exhaustion campaign, an unauthorized connection, a credential read or a privilege-escalation attempt. Zero-trust architecture supplies the principle: trust is never permanent and access follows from identity, workload state and continuous evaluation. Inside the Near-RT RIC this means combining evidence from the Kubernetes control plane, the container runtime, RIC service exposure and xApp resource behavior, then acting on it with cloud-native enforcement. Existing runtime protections commonly focus on either detection or enforcement, leaving the closed-loop question of whether a compromised xApp has been successfully removed from the control plane insufficiently addressed.

ZT-XGuard implements this with two commitments. First, detection is split across two channels with different error profiles: a behavioral channel reads kernel system calls and isolates immediately on a single high-confidence exploitation event, while a statistical channel treats a multivariate resource score as probabilistic evidence, raising suspicion and holding a fixed operator-intervention window before it isolates. Second, containment is verified, not assumed: a confirmed compromise revokes the xApp’s SPIFFE identity, quarantines it with a deny-all NetworkPolicy backed by a node-level block, zeroes its Service endpoints and probes the pod to confirm unreachability before recording containment. Restoration is gated on re-attestation, not a pod restart or a cleared alert.

Our contributions are:

- i. A dual-channel runtime monitor pairing eBPF behavioral detection with a multivariate resource-anomaly detector, fused by a policy engine into trust states for complementary coverage of exploitation and resource-abuse threats.
- ii. A verified multi-lock containment that revokes workload identity, applies micro-segmentation and an independent node-level network block, isolates service endpoints and confirms unreachability with an active reachability probe.
- iii. An evaluation on a Kubernetes Near-RT RIC testbed over 400 behavioral and 40 statistical attack trials, reporting detection latency, per-mechanism and end-to-end containment latency and effectiveness.

II. BACKGROUND AND RELATED WORK

A. Near-RT RIC and xApps

The Near-RT RIC is a programmable control layer for RAN functions, hosting xApps as containerized workloads on Kubernetes and operating over near-real-time control loops [3]. xApps consume RAN telemetry and issue control or optimization decisions through the E2 interface and RIC platform services such as RIC Message Router (RMR) messaging, the Shared Data Layer (SDL) and subscription management.

B. Related Work

Prior work secures O-RAN and cloud-native workloads at different points of the lifecycle, but leaves the running-xApp trust question open. *Admission-time access control*: xApp authentication and authorization frameworks establish who may run an xApp and what it may access when it is admitted [4]; their primary focus is admission-time authentication and authorization rather than continuous assessment of post-deployment xApp behavior. *Detection without containment*: anomaly detection for the E2 interface and xApp behavior identifies misuse [5], which is closest to our statistical channel, but stops at an alert and does not act on it. *Zero trust for O-RAN*: ZTRAN prototypes zero-trust xApps and isolation [6] and ZT-RIC applies zero-trust controls to protect RIC data privacy and confidentiality [7]; both advance zero trust in the RIC, but neither closes a verified runtime loop from detection to containment across complementary evidence channels. *Cloud-native runtime security*: prior work has studied runtime threats and container-escape attacks in Kubernetes environments [8], while general runtime tools such as Falco [9] and workload-identity systems such as SPIFFE/SPIRE [10] provide detection and identity primitives. These approaches are RIC-agnostic and do not provide an xApp-specific trust policy that continuously fuses runtime evidence with verified multi-layer containment. ZT-XGuard addresses this gap by coupling two runtime channels with different error profiles to a policy engine that fuses their evidence into a common trust state and to close the loop with containment whose enforcement is actively verified and whose restoration is gated on re-attestation, rather than stopping at admission control, alerting or ungoverned primitives.

III. THREAT MODEL

System and trust assumptions. We consider a Kubernetes-hosted Near-RT RIC where xApps run as containers beside the RIC platform. ZT-XGuard trusts the cluster control plane, the SPIRE identity infrastructure and the monitoring and policy-engine stack, as in prior cloud-native runtime-security frameworks. Defending them is out of scope: an attacker controlling the Kubernetes API server or SPIRE root can subvert any policy above them, so their integrity is a precondition. They may still deliver noisy or delayed telemetry.

Adversary capabilities. The adversary compromises a single xApp, via a subverted image or supply chain, exploitation of a vulnerable xApp or abuse of its granted permissions. We assume code execution at the xApp’s own privilege level, root inside the container but without host access, inheriting

the xApp’s RIC identity and network reachability. From there the attacker may exhaust CPU, memory or network, open unauthorized connections, execute unexpected processes, read credentials or SPIFFE key material, tamper with configuration or attempt container escape.

Scope. ZT-XGuard targets runtime resource abuse and system-level exploitation from a compromised xApp. Attacks through the xApp’s legitimate RAN-control semantics, malicious but well-formed control logic, E2 data poisoning and stealthy control-plane manipulation, are out of scope, as they need application-level semantic models rather than the system-level evidence we monitor. Container escape is handled at the boundary: the behavioral channel detects the escape attempt at the system-call level and contains the xApp before host compromise completes; we do not claim to remediate an attacker already holding host-level control, which belongs to the trusted base. Control-plane-wide denial-of-service and physical-layer radio attacks are also out of scope.

Evasion. A resource-abuse adversary may stay below thresholds or mimic benign bursts; Section VII quantifies the resulting boundary (a minimum abuse intensity and duration below which detection is declined) and the CPU-corroboration gate denies a duty-cycling attacker whose mean load looks benign. An exploitation adversary may take execution paths no rule covers, a fundamental limit of signature-style detection that the independent statistical channel offsets whenever the evasive path still consumes resources.

IV. ZT-XGUARD ARCHITECTURE

ZT-XGuard runs as a monitoring and response layer around each protected xApp (Fig. 1), feeding evidence from two independent channels into one policy engine that owns the xApp’s trust state and orchestrates containment. Each xApp carries a SPIFFE SVID issued by a SPIRE server after node and workload attestation [10]; ZT-XGuard treats this attested short-lived identity whose future issuance and renewal can be withdrawn as part of containment.

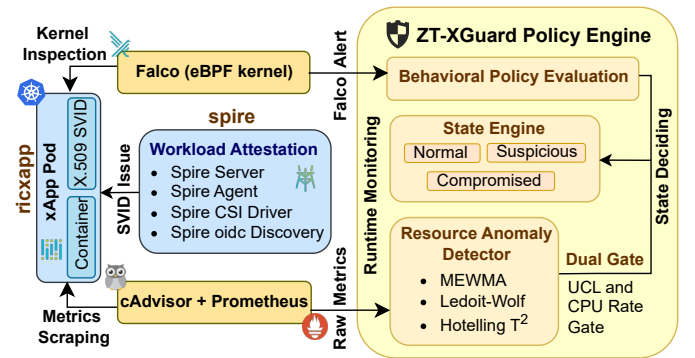


Fig. 1. ZT-XGuard detection architecture.

The *behavioral channel* observes kernel system calls and reports high-confidence exploitation events: in-container shell execution, reads of sensitive paths or SPIFFE material, unexpected outbound connections, unauthorized tool execution, configuration tampering and privilege escalation. The *statistical channel* scores resource telemetry (CPU, memory growth,

network I/O) against an xApp-specific baseline and reports a graded anomaly signal. The *policy engine* fuses the two at the decision level: each proposes a state from its own evidence and the engine holds the highest-severity proposal (NORMAL < SUSPICIOUS < COMPROMISED). A behavioral CRITICAL event overrides directly to COMPROMISED and isolates immediately; a low-severity event or a warning-band statistical signal raises SUSPICIOUS, where scrutiny rises and a reversible CPU throttle applies but no isolation occurs; a statistical anomaly confirmed by both gates escalates to COMPROMISED through a 30 s dwell; and the only exit is a verified restore to NORMAL. This late fusion keeps a single hard exploitation event decisive while preventing a probabilistic resource signal from removing an xApp alone and gives the statistical channel two roles: the multivariate score raises SUSPICIOUS and drives the early throttle, while the dual gate (persistence plus CPU corroboration) governs escalation to isolation.

V. DUAL-CHANNEL THREAT DETECTION

A. Behavioral Channel: kernel-level Exploitation Evidence

The behavioral channel uses Falco [9] to evaluate system calls captured by an eBPF probe. Per-xApp rules encode which processes, file paths and destinations are legitimate, so a rule fires on misuse rather than generic activity. These rules are generated from a declarative per-xApp security profile and synced automatically, so onboarding or updating an xApp is a profile edit rather than hand-written logic, bounding maintenance cost as the fleet grows. CRITICAL-tier rules map to confirmed compromise and isolate immediately, since a single occurrence (an in-container shell, a read of SVID key material) is already strong evidence and delay only extends the attacker's window. Evaluation is decision-synchronous: the first matching event raises the state and starts containment, with no separate scoring stage.

B. Statistical Channel: Multivariate Resource Anomaly

The statistical channel targets slow or stealthy resource abuse that triggers no exploitation event. It scores four container metrics per xApp against a frozen benign baseline: CPU usage, memory-growth rate and the network RX and TX rates ($p = 4$). Each feature is robustly standardized and one-sided rectified so that only above-baseline deviation contributes:

$$e_t = \max\left(0, \frac{g(r_t) - \tilde{m}}{1.4826 \text{ MAD}}\right), \quad (1)$$

where g applies $\log(1 + \cdot)$ to CPU and the two network rates and the identity to memory-growth, $\tilde{m}$ is the benign median, MAD is the Median Absolute Deviation and 1.4826 MAD a robust scale. Median resists the heavy-tailed bursts of legitimate RAN traffic that one-shot mean/variance calibration would absorb into the baseline. The rectified excess is averaged over an 8 s window, $x_t = \frac{1}{w} \sum_{k=t-w+1}^t e_k$ and smoothed by a multivariate exponentially weighted moving average (MEWMA) [11]:

$$Z_t = \lambda x_t + (1 - \lambda) Z_{t-1}, \quad \lambda = 1 - 2^{-1/H}, \quad (2)$$

whose recency weight λ follows from a half-life H . The smoothed state is fused into one scalar by a Hotelling T^2 statistic,

$$T_t^2 = Z_t^\top \Sigma^{-1} Z_t, \quad (3)$$

where the precision matrix Σ^{-1} inverts a Ledoit–Wolf shrinkage covariance [12], $\Sigma = \alpha F + (1 - \alpha)S$, with S the sample covariance, $F = (\text{Tr } S/p)I$ the shrinkage target and α the analytic intensity. Shrinkage keeps Σ invertible; using covariance rather than correlation preserves the 500–1000× variance gap between the CPU and network dimensions that T^2 must weight.

The channel confirms an anomaly through a *dual gate*: persistence ($T_t^2 \geq$ Upper Control Limit in six of the last eight seconds) and a CPU rate gate (over 15% of the last 60 s above a frozen 56.6 mCPU floor). Persistence rejects single-sample spikes; the rate gate defeats a duty-cycling attacker whose mean load looks benign. Both gates passing yields a confirmed anomaly mapped to COMPROMISED; a score above the lower Warning Limit (WL) without both gates is elevated and maps to SUSPICIOUS. The WL and UCL are frozen benign quantiles (10.44, 15.31). All parameters are fixed before attack evaluation; their calibration and sensitivity are discussed in Section VII-D. Because the evidence is probabilistic, the prototype applies a configurable 30 s dwell before isolation, allowing operator intervention while requiring the anomaly to remain confirmed. The 30 s value is an operational policy choice rather than a detector parameter and is therefore reported separately from detection and containment latency.

VI. VERIFIED CONTAINMENT AND GATED RESTORATION

Containment is a single, common response: whichever channel raises an xApp to COMPROMISED, the policy engine runs the same removal process and verifies it (Fig. 2). The trigger changes only what justified the action, not how the xApp is contained. Four independent containment actions apply:

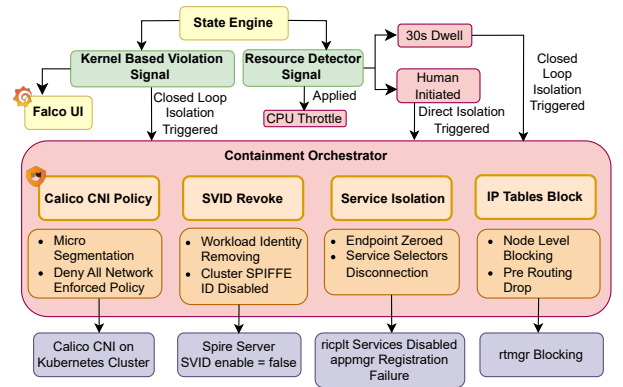


Fig. 2. ZT-XGuard containment architecture.

- 1) **Micro-segmentation.** A deny-all Kubernetes NetworkPolicy, enforced at the Container Network Interface (CNI) data plane, drops the xApp's ingress and egress. We confirmed dataplane enforcement with raw TCP-connect tests rather than assuming the policy object alone is honored.

- 2) **Identity revocation.** The xApp’s SPIFFE identity is revoked via its `ClusterSPIFFEID` binding: once disabled, SPIRE stops renewing and removes the SVID registration, so the xApp can no longer present a valid identity to any peer that checks.
- 3) **Service-endpoint isolation.** The xApp’s Service selector is swapped to a nonce, zeroing its endpoints, which cuts reachability even for a peer holding a cached endpoint.
- 4) **Direct node-level network block.** Independently of the NetworkPolicy path, ZT-XGuard installs drop rules for the xApp’s address in a dedicated chain in the node’s packet filter and confirms their presence. Because this does not depend on the CNI pipeline, the two network locks enforce the same decision through independent mechanisms, so containment never rests on a single layer.

ZT-XGuard then verifies containment: it checks endpoint membership, network-policy selection, node-level rule presence and pod readiness and runs an active TCP-connect probe from an uninvolved pod to the xApp’s Service. Only when the probe confirms unreachability is the incident recorded as contained. Restoration is conservative: an isolated xApp returns only after its identity, image digest, configuration, readiness and Service membership re-validate and ZT-XGuard recreates the pod from the clean deployment template rather than lifting restrictions on the suspect pod. A restart or a cleared alert alone does not restore RIC service.

VII. EVALUATION

A. Testbed and Methodology

Runtime detection and containment ran on the NIST 5G open-source testbed automation platform [13], integrating a 5G core (Open5GS), a srsRAN gNB and a Kubernetes Near-RT RIC [3] hosting a real E2 xApp, the KPI-monitoring xApp (kpimon-go), as the protected workload. Attacks are triggered by kernel system calls inside the live pod, so latency reflects the full path from kernel event through Falco, alert forwarding and the decision webhook; we avoid synthetic alert injection, which bypasses kernel detection. The corpus uses real tooling: resource-abuse scenarios are driven by stressing [14] at calibrated loads and container-escape and malicious-tool-execution scenarios reproduce Peirates [15] techniques, so the evaluation exercises real adversary behavior. The statistical detector uses a per-xApp benign baseline, calibrated exclusively on benign kpimon-go telemetry. Calibration data are separated from the held-out benign samples and attack traces used for evaluation; no attack samples are used to select thresholds. The behavioral corpus is eight critical-tier attacks (shell spawn, sensitive-file and SVID-material access, service-account-token access, malicious tool execution, external egress, config tamper, container-escape attempt), 50 trials each (400 total); the statistical corpus is five scenarios (benign control, sustained CPU flood, burst pulse train, stealth staircase, combined stealth), 10 trials each (50 total).

B. Behavioral-Channel Detection and Containment

ZT-XGuard contained all 400 trials across the eight attacks. The detection decision, classifying an incoming event as an

attack, takes 150 to 228 ms, a small, stable part of the response. From attack onset the reachability probe confirms network cutoff in a median 2.32 s (p95 4.20 s, p99 4.46 s) and full identity-and-state isolation, which also revokes the SVID, completes in a median 4.53 s. Fig. 3 pairs per-attack detection with this isolation latency, which runs from 4.1 s (external egress) to 5.5 s (profile tampering). The unexpected shell reaches cutoff fastest, about 1 s, as its evidence appears at the first syscall, though full isolation still takes about 5 s once identity revocation is included.

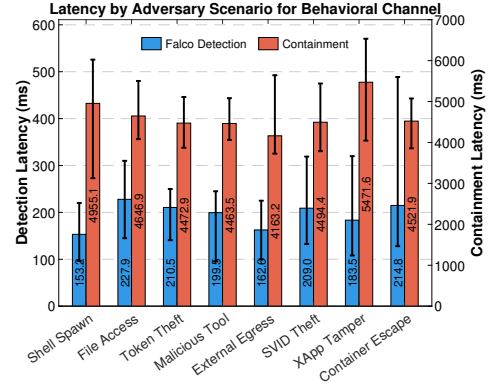


Fig. 3. Behavioral channel per attack detection latency (left axis) and detection to containment latency (right axis).

Per lock, the quarantine label, Service-endpoint isolation and direct node-level block all land within 2.3 to 2.5 s and identity revocation near 4.4 s, after which the reachability probe confirms the path is genuinely cut.

C. Statistical-Channel Detection and Enforcement

Across repeated trials during the evaluation of the statistical channel, every trial escalated to COMPROMISED for each of the four evaluated attack scenarios.

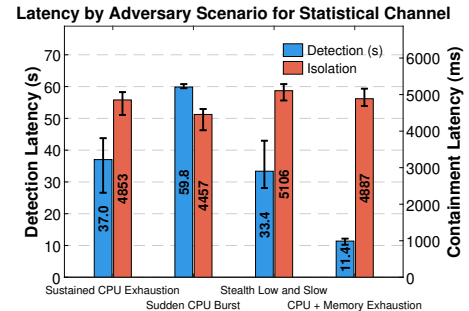


Fig. 4. Statistical channel per resource scenario: attack-onset-to-detection (left axis) and enforcement latency after the configurable 30 s dwell (right axis). The dwell is excluded from the plotted containment latency.

It never falsely contained benign activity: across 4,397 clean-phase samples over held-out benign and elevated-load runs, no sample escalated to COMPROMISED. Thus, no false containment event was observed in the evaluated benign workload. Attack-onset-to-detection is 37.0 s (sustained CPU flood), 59.8 s (burst pulse train, whose inter-pulse gaps delay six-of-eight persistence), 33.4 s (stealth staircase) and 11.4 s (combined CPU-and-memory) (Fig. 4). The combined scenario’s 11.4 s, three times faster than the staircase despite

lower per-dimension CPU, is the expected multivariate gain: co-deviating dimensions aggregate and cross the limit sooner than a single larger one, where a multivariate detector beats per-metric thresholds. Isolation then completes in 4.5 to 5.1 s; the fixed 30 s dwell preceding it is constant and omitted from Fig. 4 for clarity.

Before any isolation, an xApp at SUSPICIOUS is throttled (Fig. 5): within 16.7 to 27.2 ms of detection a reversible cgroup cap holds CPU near the 40 mCPU limit, down from 200 to 260 mCPU, real enforcement rather than an alert.

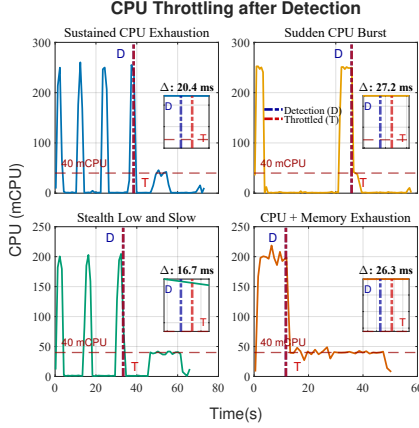


Fig. 5. CPU throttling after escalation to suspicious state, where D denotes detection and T denotes enforcement of the 40-mCPU throttle.

These latencies also fix a detection boundary. The rate gate requires elevated CPU in more than 15% of a 60 s window, so the shortest reliably detectable burst is $0.15 \times 60 = 9$ s: in the pulse-duration sweep a 5 s burst evades by design, while every burst ≥ 10 s is caught within 10–11 s, matching the 9 s bound to within one second. Detection rate stays $\geq 95\%$ down to the frozen CPU floor and degrades gracefully below it. Thus, and adversary must remain below the CPU floor or the approximately 9 s persistent bound to evade detection within the evaluated resource-abuse mode.

D. Ablation and Parameter Sensitivity

We first isolate the contribution of each detection channel. The Falco-based behavioral channel detects the evaluated rule-defined exploitation events, such as shell execution, SVID access, unauthorized egress and container-escape attempts, but does not detect resource abuse that violates no behavioral rule. Conversely, the MEWMA-based statistical channel detects the evaluated resource-abuse scenarios but does not respond to exploitation events that produce no significant resource anomaly. Their contribution therefore broadens threat coverage relative to either channel individually.

Within the statistical channel the two confirmation gates play complementary roles. The six-of-eight persistence gate rejects single-sample spikes and fixes when the detector commits, so it governs detection timing and hit volume; the CPU-corroboration gate separates sustained abuse from benign score excursions and holds the containment-tier false-positive rate at zero. Each gate addresses a different error mode, so a confirmed decision needs both. The remaining parameters come from benign calibration, not attack tuning: the CPU

floor (56.6 mCPU) is a one-sided nonparametric tolerance bound [16] over 5,400 pooled benign samples, the warning and control limits (10.44, 15.31) are high quantiles of the benign T^2 distribution on a held-out clean run, and the Ledoit–Wolf intensity is the analytic closed form ($\alpha^* = 2.5 \times 10^{-4}$). A 42-point window/half-life sweep places the frozen 8 s/3 s choice on a smooth, monotonic latency-versus-variance surface, so detection is robust to small parameter changes.

E. Scalability of the Security Functions

The results above establish correctness for one protected xApp; this subsection measures behavior as load grows. Neither the O-RAN specifications nor operator disclosures fix a standard xApp count for a Near-RT RIC, and reported configurations range from single-figure reference workflows, such as the five-xApp O-RAN Software Community traffic-steering suite [17] and two-xApp testbed studies [18], to infrastructure-scaling work that grows a lightweight xApp from 38 to 89 pods [19]. We make no xApp-hosting claim and instead stress each of ZT-XGuard’s three security functions on the resource it can saturate.

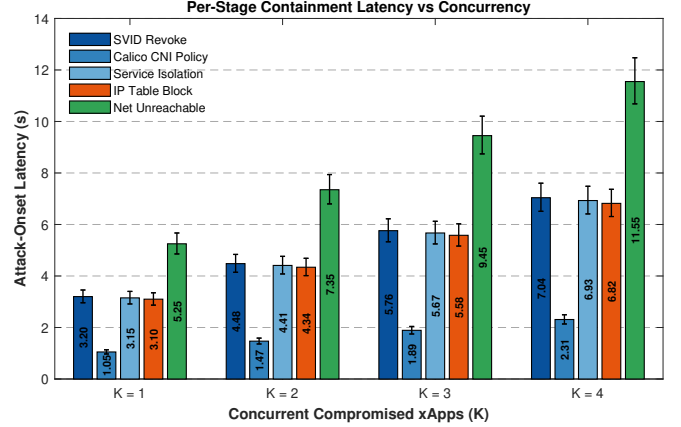


Fig. 6. Per-stage containment latency versus the number K of simultaneously compromised xApps (mean over 100 trials per level).

Detector compute. We replicate the trust detector into N independent contexts that share one trained model but keep separate MEWMA, T^2 and trust state, and rescore all N once per second. Over a sweep to $N=80$ and 346,500 scoring invocations, no cycle missed the 1 s budget; at $N=80$ the median cycle was 3.6 ms and the worst 50.7 ms, near 5% of the budget, and each context produced the same trust sequence alone or concurrently.

Decision plane. Under open-loop offered load the policy path held a zero deadline-miss rate through 20 evaluations per second and 1.96% at 25, then saturated to 33% misses at 30 and 98% at 80, with no request errors or process restarts at any rate. Its single-core operating envelope is about 20 to 25 evaluations per second; the knee is set by CPU and would rise with horizontal replication of the stateless decision endpoint.

Concurrent enforcement. Containment is exercised hardest when several xApps are compromised at once, so we inject a controlled in-container attack into K of the four deployed xApps simultaneously ($K=1$ to 4, 100 trials each, launch

skew under 2 ms) and run the full four-lock pipeline to a probe-verified unreachable state. Every trial isolated its targets. The containment makespan, from attack onset to verified unreachability, grew from 5.25 s at $K=1$ to 11.55 s at $K=4$ (Fig. 6), so latency rises with concurrency while every target is still contained.

These are single-node operating envelopes for the security functions, not a hosting-capacity claim for the RIC; multi-node placement and a queue-backed evaluator remain future work.

F. Containment Effectiveness and Overhead

The locks stop new traffic, but the meaningful outcome, whatever the trigger, is that the xApp drops out of the RIC control loop (Fig. 7): after isolation its periodic E2 RIC-indication traffic stops, received throughput falls by about half, and the reachability probe confirms full unreachability within about 4 s, directly verifying removal from the Near-RT RIC service path.

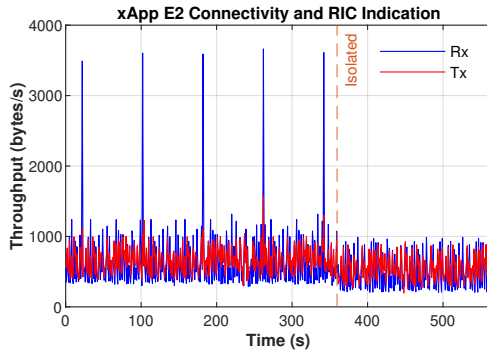


Fig. 7. xApp E2 traffic before and after containment: the periodic RIC indication peaks stop after isolation.

Monitoring overhead. Steady-state cost is small relative to the Near-RT RIC budget. Across the 400 trials the Falco eBPF monitor used a median 12.4% of one core (max 16.2%) and about 109 MB resident, dropped zero events even under attack load. Measured against a vanilla RIC over a 15-minute steady-state window, the policy engine adds about 0.004 of a core to the node mean and a fixed 98 MB of memory, with a sub-second CPU spike only during containment. Outside containment events, the policy engine itself adds negligible additional CPU load beyond the continuous Falco monitoring overhead reported above.

VIII. CONCLUSION AND FUTURE WORK

ZT-XGuard is a zero-trust framework for the O-RAN Near-RT RIC that fuses a kernel-behavioral and a multivariate statistical channel into one trust decision and closes the loop with verified, multi-lock containment and re-attested restoration. It contained all 400 behavioral trials and detected every resource-abuse scenario with no false isolations. Its scope leaves cross-xApp generalization open, since the false-positive rate is measured on one xApp's benign telemetry. Future work will learn per-xApp baselines automatically for heterogeneous fleets, extend detection to RAN-application-level attacks such as control-logic manipulation and E2 data poisoning, and evaluate a multi-node deployment at operator scale.

ACKNOWLEDGMENT

The authors used a large language model (Anthropic Claude) for language editing of the manuscript based on the original research work and take full responsibility for the content.

REFERENCES

- [1] M. Polese, L. Bonati, S. D'Oro, S. Basagni, and T. Melodia, "Understanding O-RAN: Architecture, interfaces, algorithms, security, and research challenges," *IEEE Commun. Surveys Tuts.*, vol. 25, no. 2, pp. 1376–1411, 2023.
- [2] O-RAN Alliance, "Study on security for near real time RIC and xApps, O-RAN.WG11.TR.Security-Near-RT-RIC-xApps.0-R004-v06.00," O-RAN Alliance, Technical Report, 2025.
- [3] —, "Near-RT RIC architecture, O-RAN.WG3.TS.RICARCH-R004-v07.00," O-RAN Alliance, Technical Specification, 2025.
- [4] P. Fernando, P. Porambage, M. Liyanage, K. Steenhaut, and A. Braeken, "Securing xApps in open RAN: A hierarchical approach to authentication and authorisation," in *Proc. IEEE Conf. Commun. Netw. Secur. (CNS)*, 2025.
- [5] C.-F. Hung, C.-H. Tseng, and S.-M. Cheng, "Anomaly detection for mitigating xApp and E2 interface threats in O-RAN near-RT RIC," *IEEE Open J. Commun. Soc.*, 2025.
- [6] A. S. Abdalla, J. Moore, N. Adhikari, and V. Marojevic, "ZTRAN: Prototyping zero trust security xApps for open radio access network deployments," *IEEE Netw.*, 2024.
- [7] D. Lin, S. Bhargav, A. Chiejina, M. I. Ibrahim, and V. K. Shah, "ZT-RIC: A zero trust RIC framework for ensuring data privacy and confidentiality in open RAN," in *Proc. IEEE Consum. Commun. Netw. Conf. (CCNC)*, 2025.
- [8] J.-N. Luo, W.-H. Zou, and S.-W. Huang, "From container to cluster: Chained escape attacks in kubernetes and orchestration platforms," *IEEE Access*, vol. 13, pp. 188 217–188 235, 2025.
- [9] The Falco Project (CNCF), "Falco: Cloud-native runtime security," accessed: Aug. 29, 2026. [Online]. Available: <https://falco.org>
- [10] SPIFFE Project (CNCF), "The SPIFFE/SPIRE workload identity framework," accessed: Aug. 29, 2026. [Online]. Available: <https://spiffe.io>
- [11] C. A. Lowry, W. H. Woodall, C. W. Champ, and S. E. Rigdon, "A multivariate exponentially weighted moving average control chart," *Technometrics*, vol. 34, no. 1, pp. 46–53, 1992.
- [12] O. Ledoit and M. Wolf, "A well-conditioned estimator for large-dimensional covariance matrices," *J. Multivariate Anal.*, vol. 88, no. 2, pp. 365–411, 2004.
- [13] National Institute of Standards and Technology (NIST), "NIST 5G open-source testbed automation tool," accessed: Aug. 29, 2026. [Online]. Available: <https://www.nist.gov/services-resources/software/nist-5g-open-source-testbed-automation-tool>
- [14] C. I. King, "stress-ng: a tool to load and stress a computer system," accessed: Aug. 30, 2026. [Online]. Available: <https://github.com/ColinIanKing/stress-ng>
- [15] InGuardians. Peirates: Kubernetes penetration testing tool. Accessed: Aug. 30, 2026. [Online]. Available: <https://github.com/inguardians/peirates>
- [16] S. S. Wilks, "Determination of sample sizes for setting tolerance limits," *Ann. Math. Statist.*, vol. 12, no. 1, pp. 91–96, 1941.
- [17] O-RAN Software Community, "Traffic steering use case (xApp) user guide," O-RAN Software Community documentation, accessed: Aug. 29, 2026.
- [18] A. Sultana *et al.*, "Experimental evaluation of xApp conflict mitigation framework in O-RAN: Insights from testbed deployment in OTIC," in *Proc. IEEE INFOCOM Wkshps*, 2025.
- [19] S. Kumar, R. Zitouni, A. Akbas, and C. H. Foh, "FlexScale: Scalable and efficient management approach for near-RT RIC in O-RAN," in *Proc. IEEE Wireless Commun. Netw. Conf. (WCNC)*, 2025.